Data Analytics Using SQL - Question Bank with Answers

Subqueries, Derived Tables and CTEs; Stored Procedures, Functions and Triggers; Window Functions

Part A - 2 Marks

Q1. What is a subquery? State the three types of subqueries.

Introduction: A subquery is a query nested inside another SQL query; it supplies a value or a set of values that the outer (main) query uses to filter or calculate.

- **Definition and use:** A subquery is written inside parentheses and is normally placed in the WHERE, FROM or SELECT clause. It lets an analyst isolate one calculation, such as a maximum or an average, and reuse the result in the main query.
- **Types:** A single-row subquery returns one value and is used with =, < and >. A multi-row subquery returns many values and is used with IN, ANY or ALL. A correlated subquery refers to outer-query columns and runs once for every outer row.

Q2. Define a derived table. State one benefit of using it.

Introduction: A derived table is a temporary result set created by a subquery written in the FROM clause, which the outer query then treats like an ordinary table.

- **Definition:** It exists only while the query runs and must be given an alias, for example FROM (SELECT Department, Salary FROM Employee WHERE Salary > 50000) AS high_paid. The outer query refers to its columns through that alias.
- **Benefit:** It breaks a complex query into small, manageable steps and improves readability, because filtering or aggregation is completed first in the inner part and the outer query works on the prepared result.

Q3. What is a Common Table Expression (CTE)? Write its basic syntax.

Introduction: A CTE is a temporary, named result set defined with the WITH clause that can be referred to within a single SELECT, INSERT, UPDATE or DELETE statement.

```
WITH HighPaid AS (
    SELECT Name, Salary
    FROM Employee
    WHERE Salary > 50000
)
SELECT Name, Salary
FROM HighPaid;
```

- **Syntax:** WITH cte_name AS (query) defines the named result set, and the statement that follows reads from cte_name like a table. Several CTEs can be chained in one WITH clause by separating them with commas.
- **Benefit:** A CTE improves readability, reusability and modularity. The same CTE can be referenced more than once, and each step can be tested separately, which makes debugging easier than with deeply nested subqueries.

Q4. Write an SQL query using a subquery to display the employees whose salary is greater than the average salary of the table Employee(EmpID, Name, Department, Salary).

Introduction: A single-row (scalar) subquery returns one value, here the average salary, and the outer query uses that value in its WHERE condition.

```
SELECT EmpID, Name, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

- **Working:** The inner query SELECT AVG(Salary) is executed first and returns a single number. The outer query then keeps only those rows whose Salary is greater than that number.

- **Expected output:** For the sample data (Asha 60000, Ravi 45000, Meera 52000, Karan 38000, Divya 70000, Imran 48000, Neha 75000) the average is about 55428.57, so three employees qualify.

```
EmpID  Name   Salary
1      Asha   60000
5      Divya  70000
7      Neha   75000
```

Q5. Define formal and actual parameters in a stored procedure with a suitable example.

Introduction: Parameters pass values between a calling program and a stored procedure. Formal parameters are declared in the procedure header, while actual parameters are the values supplied when the procedure is called.

```
PROCEDURE cal_bonus(emp_id IN NUMBER, rate IN NUMBER, bonus OUT NUMBER) IS
...
-- Calling the procedure
cal_bonus(current_emp_id, 10, current_bonus);
```

- **Formal parameters:** emp_id, rate and bonus are the formal parameters. Each is declared with a name, a mode (IN, OUT or IN OUT) and a datatype, and acts as a placeholder inside the procedure body.
- **Actual parameters:** current_emp_id, 10 and current_bonus are the actual parameters. They are the real variables or values placed in the call, and they are matched to the formal parameters by position.

Q6. What is a database trigger? State one use of triggers.

Introduction: A trigger is a named PL/SQL program unit stored in the database that executes (fires) automatically when an INSERT, UPDATE or DELETE operation occurs on a table or view.

- **Characteristics:** A trigger cannot accept input parameters and cannot be called explicitly. It runs only when its triggering event takes place, and its execution is transparent to the user who issued the statement.
- **Use:** Triggers enforce complex business rules, supplement referential integrity and audit data changes, for example by copying the old row into an audit table whenever a salary is updated.

Q7. What is a window function? State the role of the OVER clause.

Introduction: A window function performs a calculation on a set (window) of rows related to the current row and returns a value for every row, without collapsing the rows as GROUP BY does.

- **Window function:** The SQL standard supports aggregate (SUM, AVG), ranking (RANK, ROW_NUMBER), distribution (PERCENT_RANK) and offset (LAG, LEAD) functions. Because detail rows are retained, detail and aggregate values can appear together.
- **OVER clause:** OVER defines the window for each row. PARTITION BY splits the rows into groups, ORDER BY sets the order inside a group, and a frame limits the rows used. An empty OVER() means the whole result set.

Q8. Write a query using LAG() to display each month's revenue, the previous month's revenue and the change from MonthlySales(SaleMonth, Revenue).

Introduction: LAG() is an offset window function that returns a value from a preceding row, so month-over-month change can be found without a self-join.

```
SELECT SaleMonth, Revenue,
       LAG(Revenue, 1) OVER (ORDER BY SaleMonth) AS PrevRevenue,
       Revenue - LAG(Revenue, 1) OVER (ORDER BY SaleMonth) AS Change
FROM MonthlySales;
```

- **Working:** LAG(Revenue, 1) OVER (ORDER BY SaleMonth) fetches the revenue of the row one position earlier in month order. The first month has no earlier row, so it returns NULL.

- **Expected output:** For January 40000, February 46000 and March 43000 the result is shown below; the Change column is NULL for the first month.

```
SaleMonth  Revenue  PrevRevenue  Change
2026-01    40000    NULL         NULL
2026-02    46000    40000        6000
2026-03    43000    46000        -3000
```

Q9. What is an index? Name any two types of indexes.

Introduction: An index is a data structure that speeds up data retrieval from a table by letting the database locate rows without scanning every row.

- **Working:** The database builds a separate structure that points to the table rows, just like the index of a book. Reads become faster, but extra storage is used and INSERT, UPDATE and DELETE become slightly slower.
- **Types:** A B-tree index uses a balanced tree and suits both equality and range queries. A hash index suits only equality comparisons. Bitmap indexes (low-cardinality columns) and full-text indexes are other types.

Q10. State any two steps involved in performance tuning of a database.

Introduction: Performance tuning is the process of optimizing database operations to reduce query response time and improve overall efficiency.

- **Identify and analyse slow queries:** The longest-running queries are found first, and the EXPLAIN command is used to read their execution plans. This shows whether a query uses an index scan or a costly full table scan.
- **Use indexes and fetch only required data:** Indexes (including composite indexes) are added on columns used in WHERE and JOIN conditions, and queries select only the needed columns and rows to reduce processing and data transfer.

Part B - 5 Marks

Q11. Differentiate between single-row, multi-row and correlated subqueries with examples.

Introduction: Subqueries are classified by the number of values they return and by their dependence on the outer query; the differences are given below.

Basis	Single-row subquery	Multi-row subquery	Correlated subquery
Values returned	Exactly one value	Many values (a list)	One or many values for each outer row
Operators used	=, <, >, <=, >=, <>	IN, ANY, ALL	Any operator, often with >, = or EXISTS
Execution	Executed once, before the outer query	Executed once, before the outer query	Executed once for every row of the outer query
Dependence on outer query	Independent	Independent	Refers to outer-query columns (for example e1.Department)
Typical use	Compare with a maximum or average	Match against a list of values	Compare a row with a group figure such as the department average

```
-- Single-row
SELECT Name FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);

-- Multi-row
SELECT Name FROM Employee
WHERE Salary IN (SELECT MAX(Salary) FROM Employee GROUP BY Department);

-- Correlated
SELECT Name, Salary FROM Employee e1
WHERE Salary > (SELECT AVG(Salary) FROM Employee e2
                WHERE e2.Department = e1.Department);
```

Q12. Differentiate between a stored procedure and a function in PL/SQL with examples.

Introduction: Both are named program units stored in the database, but they differ in what they return and where they can be used.

Basis	Procedure	Function
Return value	Can return multiple output values through OUT or IN OUT parameters	Returns a single value using the RETURN statement
Parameter modes	IN, OUT and IN OUT are all allowed	Normally IN only; an inline function must use only IN parameters
Use in SQL statements	Cannot be used inside a SELECT statement	Can be used as an inline function in SELECT if it meets the purity rules
Calling method	EXECUTE name(...) in SQL*Plus or by a plain call from another program unit	Called as part of an expression, for example v := name(...) or inside a query
Purpose	Performs an action such as inserting or updating records	Computes and returns a value such as age or balance

```
-- Procedure: performs an action
EXECUTE add_worker('Adah Talbot');

-- Function: returns a value, used inline in a query
SELECT Name, service_years(HireDate) FROM Employee;
```

Q13. Explain derived tables with a suitable example using the table Employee(EmpID, Name, Department, Salary).

Introduction: A derived table is a temporary result set produced by a subquery in the FROM clause, which the outer query references through an alias as if it were a real table.

- **Definition:** A derived table is created and used within one query and is discarded when the query finishes. It is not stored in the database, so it needs no CREATE statement and leaves no object behind.
- **Alias rule:** The subquery must be followed by an alias (AS name). The outer query uses this alias to refer to the derived columns, and the alias also makes the structure of a long query easier to follow.
- **Benefits:** It divides a complex query into logical segments, improves readability and allows a calculation, such as an aggregate, to be performed in two stages: first filter or summarise, then analyse the prepared set.

- **Example:** The query below first keeps employees earning more than 50000 and then finds the average salary of that group per department. For the sample data, Finance has only Divya (70000), HR only Meera (52000), and IT has Asha and Neha.

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM (SELECT Department, Salary
      FROM Employee
      WHERE Salary > 50000) AS high_salary_employees
GROUP BY Department
ORDER BY Department;

Department  AvgSalary
Finance     70000.00
HR          52000.00
IT          67500.00
```

- **Use with window functions:** A window function result cannot be filtered in WHERE of the same query. Placing the ranking query in a derived table lets the outer query use WHERE rnk <= 3 to obtain the top three products per category.

Q14. Explain the different types of triggers in PL/SQL with an example.

Introduction: Triggers are classified by the statement that fires them, the time at which they fire and the level at which they fire.

- **Statement type:** A trigger can be defined for INSERT, UPDATE or DELETE. Several events can be combined with OR, and UPDATE OF column fires the trigger only when the named column is changed.
- **Timing:** A BEFORE trigger fires before the triggering statement is executed and is useful for validation or setting values. An AFTER trigger fires after the statement and is useful for auditing or updating other tables.
- **Level:** A row-level trigger, created with FOR EACH ROW, fires once for every affected row and can read :OLD and :NEW values. A statement-level trigger fires once per statement, however many rows are affected.
- **INSTEAD OF trigger:** It tells Oracle what to do instead of the original action, mainly on views. For example, it can redirect an insert to another table or update several base tables that make up a view.
- **Valid combinations:** Together these give 14 valid configurations, such as BEFORE INSERT row, AFTER UPDATE statement and INSTEAD OF row. Predicates INSERTING, UPDATING and DELETING allow one trigger to handle several events.

Example: a row-level BEFORE UPDATE trigger that stores an audit record when a salary rises by more than 10 percent.

```
CREATE OR REPLACE TRIGGER trg_salary_audit
BEFORE UPDATE OF Salary ON Employee
FOR EACH ROW
WHEN (NEW.Salary > OLD.Salary * 1.10)
BEGIN
    INSERT INTO Salary_Audit (EmpID, OldSalary, NewSalary, ChangedOn)
    VALUES (:OLD.EmpID, :OLD.Salary, :NEW.Salary, SYSDATE);
END;
/
```

Q15. Explain the ranking window functions ROW_NUMBER(), RANK(), DENSE_RANK() and NTILE() with an example.

Introduction: Ranking functions assign a position number to each row within a window according to the ORDER BY specification of the OVER clause.

- **ROW_NUMBER():** Gives a unique sequential number (1, 2, 3, ...) to each row in the partition. Rows with equal values still receive different numbers, so a tie-breaker column should be added to the ORDER BY.
- **RANK():** Gives tied rows the same rank and then skips the following numbers. The rank of a row is one more than the number of rows with a better value, so two rows at rank 2 are followed by rank 4.

- **DENSE_RANK():** Also gives tied rows the same rank but leaves no gaps in the sequence. After two rows at rank 2, the next distinct value receives rank 3, which is useful for top-N lists.
- **NTILE(n):** Divides the ordered rows into n nearly equal groups and returns the group number for each row. NTILE(4) is commonly used to create quartiles, such as top 25 percent of customers.
- **Syntax and example:** Ranking functions need ORDER BY inside OVER, and PARTITION BY restarts the numbering for every group. The query below ranks students of Marks(Student, Score) with Score values 90, 85, 85 and 70.

```
SELECT Student, Score,
       ROW_NUMBER() OVER (ORDER BY Score DESC, Student) AS RowNum,
       RANK()       OVER (ORDER BY Score DESC)          AS RankNo,
       DENSE_RANK() OVER (ORDER BY Score DESC)          AS DenseRankNo,
       NTILE(2)     OVER (ORDER BY Score DESC, Student) AS Tile
FROM Marks;

Student  Score  RowNum  RankNo  DenseRankNo  Tile
A        90     1       1       1            1
B        85     2       2       2            1
C        85     3       2       2            2
D        70     4       4       3            2
```

Q16. Explain the advantages of window functions over GROUP BY queries and subqueries, with an example.

Introduction: A window function applies a calculation to a set of rows defined by the OVER clause and returns a value for every row, so detail and aggregate information can be shown together.

- **Detail is not lost:** A grouped query returns one row per group and hides the individual rows. A window function keeps every detail row and adds the aggregate beside it, for example each order next to the customer total.
- **Simpler code:** Mixing detail and aggregates with GROUP BY needs extra table expressions and joins, and with subqueries needs one subquery per calculation. SUM(val) OVER (PARTITION BY custid) does the same work in one expression.
- **Better optimization:** SQL Server can detect several functions that share the same window specification and calculate them in one visit to the data, whereas every separate subquery normally causes another visit.
- **Filters apply once:** The window is formed from the result set after FROM, WHERE, GROUP BY and HAVING. A single WHERE filter therefore applies to all window calculations, but each subquery must repeat the filter.
- **Flexible windows:** OVER() uses the whole result set, PARTITION BY restricts it to a group, and ORDER BY with a frame supports running totals. The query below shows each sale with its region total and percentage share.

```
SELECT SaleID, Region, Amount,
       SUM(Amount) OVER (PARTITION BY Region) AS RegionTotal,
       ROUND(100.0 * Amount / SUM(Amount) OVER (PARTITION BY Region), 2) AS PctOfRegion
FROM RegionSales;

SaleID  Region  Amount  RegionTotal  PctOfRegion
1       North   2000    5000         40.00
2       North   3000    5000         60.00
3       South   4000    10000        40.00
4       South   1000    10000        10.00
5       South   5000    10000        50.00
```

Q17. Explain packages in PL/SQL. Describe the package specification, package body, overloading and built-in packages.

Introduction: A package is a database object that groups related procedures, functions, variables and cursors into a single unit, which can be shared with other database users.

- **Specification and body:** The specification declares the public items (variables, cursors and the headers of procedures and functions). The body contains the actual code of the modules and may include private items that are visible only inside the package.

```
CREATE OR REPLACE PACKAGE emp_pkg IS
    PROCEDURE raise_salary(p_emp_id NUMBER, p_pct NUMBER);
    FUNCTION  get_salary(p_emp_id NUMBER) RETURN NUMBER;
END emp_pkg;
/
CREATE OR REPLACE PACKAGE BODY emp_pkg IS
    PROCEDURE raise_salary(p_emp_id NUMBER, p_pct NUMBER) IS
    BEGIN
        UPDATE employee SET salary = salary * (1 + p_pct / 100)
        WHERE emp_id = p_emp_id;
        COMMIT;
    END;
    FUNCTION get_salary(p_emp_id NUMBER) RETURN NUMBER IS
        v_sal NUMBER;
    BEGIN
        SELECT salary INTO v_sal FROM employee WHERE emp_id = p_emp_id;
        RETURN v_sal;
    END;
END emp_pkg;
/
```

- **Calling and privileges:** A packaged item is called with the package name first, for example EXECUTE emp_pkg.raise_salary(101, 10). Other users need GRANT EXECUTE ON emp_pkg TO username before they can use it.
- **Overloading:** Several program units in a package can share one name if their parameters differ in number, order or datatype family. Differences only in parameter names or only in return type are not allowed.

```
PROCEDURE add_enrollment(p_s_id NUMBER, p_sec_id NUMBER);
PROCEDURE add_enrollment(p_first VARCHAR2, p_last VARCHAR2, p_sec_id NUMBER);
```

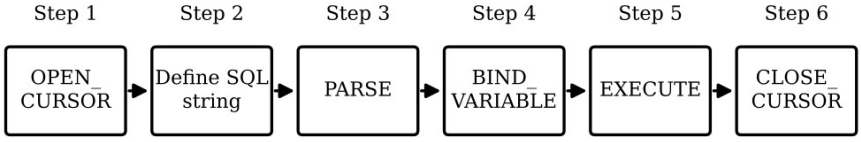
- **Advantages:** Packages allow modules to share variables and cursors, hide implementation details, keep related code together and can run initialisation code each time the package is first called, which improves organisation and reuse.
- **Built-in packages:** Oracle supplies ready-made packages such as DBMS_OUTPUT (display text), DBMS_JOB (schedule programs), DBMS_PIPE (send messages between sessions), DBMS_SQL (dynamic SQL) and DBMS_DDL (recompile program units).

Q18. Explain dynamic SQL in PL/SQL. Describe the steps of the DBMS_SQL package for DML, DDL and SELECT statements.

Introduction: Dynamic SQL builds SQL commands as text strings that are compiled and validated at run time, instead of at compile time as in static SQL.

- **Static versus dynamic:** In static SQL the command structure and the objects are checked when the program is compiled. Dynamic SQL lets a program change its query according to user input or the current date and also lets it run DDL such as CREATE, ALTER and DROP.
- **DBMS_SQL package:** All dynamic SQL is processed through a cursor, which is a server memory area. DBMS_SQL provides OPEN_CURSOR, PARSE, BIND_VARIABLE, DEFINE_COLUMN, EXECUTE, FETCH_ROWS, COLUMN_VALUE, VARIABLE_VALUE and CLOSE_CURSOR.

- **DML statements:** The steps are: open the cursor, write the command as a string with placeholders such as :pct, parse it, bind values to the placeholders, execute it and close the cursor. EXECUTE returns the number of rows processed.



DDL (CREATE / ALTER / DROP): steps 1, 2, 3, 6 only - PARSE runs the statement; no BIND or EXECUTE

```
CREATE OR REPLACE PROCEDURE raise_dept_salary(p_pct IN NUMBER, p_dept IN VARCHAR2) AS
    cursor_id    NUMBER;
    update_stmt  VARCHAR2(1000);
    rows_updated NUMBER;
BEGIN
    cursor_id   := DBMS_SQL.OPEN_CURSOR;
    update_stmt := 'UPDATE employee SET salary = salary * (1 + :pct) ' ||
                   'WHERE department = :dept';
    DBMS_SQL.PARSE(cursor_id, update_stmt, DBMS_SQL.V7);
    DBMS_SQL.BIND_VARIABLE(cursor_id, ':pct',  p_pct);
    DBMS_SQL.BIND_VARIABLE(cursor_id, ':dept', p_dept);
    rows_updated := DBMS_SQL.EXECUTE(cursor_id);
    DBMS_SQL.CLOSE_CURSOR(cursor_id);
END;
/
EXECUTE raise_dept_salary(0.10, 'IT');   -- IT salaries increase by 10 percent
```

- **DDL statements:** Placeholders cannot be used in DDL, so values are joined into the command string, and PARSE itself executes the statement. Therefore BIND_VARIABLE and EXECUTE are skipped and the steps are only open, define, parse and close.
- **SELECT statements:** After parsing and binding, DEFINE_COLUMN declares the output variables and EXECUTE runs the query. A loop then calls FETCH_ROWS until it returns 0, and COLUMN_VALUE copies each fetched column into its variable.

Part C - 10 Marks

Q19. A company stores its data in Employee(EmpID, Name, DeptID, Salary) and Department(DeptID, DeptName, Location). Design SQL queries using subqueries and derived tables to (i) find the highest-paid employee, (ii) list employees working in departments located in Mumbai, (iii) find employees earning more than every employee of department 20, (iv) find employees earning above the average salary of their own department, and (v) list departments whose average salary is above 50000. Explain how each subquery is executed.

Introduction: Subqueries and derived tables allow multi-step calculations to be written as a single, well-organised SQL statement. The sample data assumed for the answers is given below.

```
Employee: (1,Asha,10,60000) (2,Ravi,10,45000) (3,Meera,20,52000) (4,Karan,20,38000)
          (5,Divya,30,70000) (6,Imran,30,48000) (7,Neha,10,75000)
Department: (10,IT,Mumbai) (20,HR,Pune) (30,Finance,Mumbai)
```

- **Role of subqueries:** A subquery is a query nested inside another query. It isolates one calculation, such as a maximum, a list of IDs or a group average, so that the main query can use the result without any manual intermediate step.
- **Placement and nesting:** A subquery can appear in the WHERE or HAVING clause, in the FROM clause (a derived table) or in the SELECT list. Subqueries can also be nested several levels deep, for example VIP customers inside orders inside a product query.

- **(i) Single-row subquery:** The inner query returns one value, the maximum salary, and the outer query selects the employee having exactly that salary. The inner query runs once, before the outer query.

```
SELECT Name, Salary FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);
-- Output: Neha  75000
```

- **(ii) Multi-row subquery with IN:** The inner query returns the IDs of all departments located in Mumbai, and IN tests whether each employee's DeptID is in that list. IN is used because more than one value comes back.

```
SELECT Name FROM Employee
WHERE DeptID IN (SELECT DeptID FROM Department WHERE Location = 'Mumbai');
-- Output: Asha, Ravi, Divya, Imran, Neha
```

- **(iii) ALL and ANY operators:** The inner query returns every salary of department 20 (52000 and 38000). Salary > ALL means greater than the largest of them. With ANY, the condition would hold if it is greater than at least one value.

```
SELECT Name, Salary FROM Employee
WHERE Salary > ALL (SELECT Salary FROM Employee WHERE DeptID = 20);
-- Output: Asha 60000, Divya 70000, Neha 75000
```

- **(iv) Correlated subquery:** The inner query refers to e1.DeptID of the outer row, so it is re-executed for every employee to compute that employee's department average. Only rows above that average, such as Neha, Meera and Divya, are returned.

```
SELECT Name, DeptID, Salary
FROM Employee e1
WHERE Salary > (SELECT AVG(Salary) FROM Employee e2
                WHERE e2.DeptID = e1.DeptID);
-- Output: Meera 20 52000, Divya 30 70000, Neha 10 75000
```

- **Order of execution:** An uncorrelated subquery is evaluated once and its result is reused, whereas a correlated subquery runs once per outer row and can be slow on large tables. Such queries can often be rewritten with a join or a window function.
- **(v) Derived table:** The subquery in the FROM clause calculates the average salary of every department and is given the alias t. The outer query joins it to Department and keeps only averages above 50000.

```
SELECT d.DeptName, t.AvgSalary
FROM (SELECT DeptID, AVG(Salary) AS AvgSalary
      FROM Employee GROUP BY DeptID) AS t
JOIN Department d ON t.DeptID = d.DeptID
WHERE t.AvgSalary > 50000;
-- Output: IT 60000, Finance 59000
```

- **Combining a subquery and a derived table:** To list employees of the department with the highest total payroll, a derived table finds the top department by total salary and an outer subquery matches DeptID. Here IT has the highest total, 180000.

```
SELECT Name FROM Employee
WHERE DeptID = (SELECT DeptID
                FROM (SELECT DeptID, SUM(Salary) AS Total
                      FROM Employee GROUP BY DeptID
                      ORDER BY Total DESC LIMIT 1) AS top_dept);
-- Output: Asha, Ravi, Neha
```

- **Best practices:** Avoid overly complex nesting, use indexed columns inside subqueries, give derived tables meaningful names, and move repeated logic into a CTE. Prefer IN or EXISTS carefully, because NOT IN returns no rows if the list contains NULL.

Q20. Explain Common Table Expressions (CTEs) with their syntax and benefits. Using Sales(SaleID, CustomerID, Product, Category, SaleDate, Amount) and Employee(EmpID, Name, ManagerID), write CTE queries to (i) list products whose total sales exceed 50000, (ii) find the top product of each category using chained CTEs, and (iii) display the reporting hierarchy using a recursive CTE. Also state the best practices.

Introduction: A Common Table Expression is a temporary, named result set defined by the WITH clause and used within one statement; it makes multi-step analytical queries easier to write and read. Sample Sales data:

```
SaleID  CustomerID  Product  Category     SaleDate    Amount
1       C1          Laptop   Electronics  2026-01-10  60000
2       C2          Laptop   Electronics  2026-02-12  50000
3       C1          Phone    Electronics  2026-01-15  40000
4       C3          Phone    Electronics  2026-03-05  55000
5       C2          Shirt    Clothing     2026-01-20   4000
6       C4          Shirt    Clothing     2026-02-18   5000
7       C1          Jeans    Clothing     2026-03-09   9500
8       C3          Mixer    Appliances   2026-02-25  12000
9       C4          Fan      Appliances   2026-03-14   7000
10      C2          Fan      Appliances   2026-03-20   6000
```

- **Definition and syntax:** A CTE starts with WITH name AS (query) and is followed by the main statement that reads from name. It exists only for that statement and can be used with SELECT, INSERT, UPDATE or DELETE.

```
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

- **Benefits:** A CTE improves readability by splitting a complex query into logical steps, supports reusability because the same CTE can be referenced several times, and gives modularity, so each part can be debugged separately.
- **(i) Simple CTE:** ProductSales first calculates the total sales per product, and the outer query filters the totals. The aggregation and the filter are written as two clear steps instead of one nested statement.

```
WITH ProductSales AS (
    SELECT Product, Category, SUM(Amount) AS TotalSales
    FROM Sales
    GROUP BY Product, Category
)
SELECT Product, TotalSales
FROM ProductSales
WHERE TotalSales > 50000;

Product  TotalSales
Laptop   110000
Phone    95000
```

- **(ii) Chained CTEs:** Several CTEs are defined in one WITH clause separated by commas, and a later CTE can use an earlier one. RankedProducts ranks the totals produced by ProductSales inside each category.

```
WITH ProductSales AS (
    SELECT Product, Category, SUM(Amount) AS TotalSales
    FROM Sales GROUP BY Product, Category
),
RankedProducts AS (
    SELECT Product, Category, TotalSales,
           RANK() OVER (PARTITION BY Category ORDER BY TotalSales DESC) AS rnk
    FROM ProductSales
)
SELECT Category, Product, TotalSales
FROM RankedProducts
WHERE rnk = 1;

Category     Product  TotalSales
Appliances   Fan      13000
Clothing     Jeans    9500
Electronics  Laptop   110000
```

- **CTE versus derived table:** A derived table is written inside the FROM clause and can be used only once in that place, while a CTE is declared first and may be referred to many times. Both are temporary and neither is stored in the database.
- **Recursive CTE structure:** A recursive CTE, written WITH RECURSIVE, is used for hierarchical data. It has an anchor member (the starting rows), UNION ALL, and a recursive member that joins back to the CTE until no new rows are produced.
- **(iii) Reporting hierarchy:** The anchor selects the employee with no manager (level 1). Each recursive step joins Employee to the rows found so far and adds 1 to the level, so the organisation is built from top to bottom.

```
WITH RECURSIVE OrgHierarchy AS (
    SELECT EmpID, Name, ManagerID, 1 AS Level
    FROM Employee WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmpID, e.Name, e.ManagerID, oh.Level + 1
    FROM Employee e JOIN OrgHierarchy oh ON e.ManagerID = oh.EmpID
)
SELECT EmpID, Name, ManagerID, Level FROM OrgHierarchy ORDER BY Level, EmpID;

EmpID  Name    ManagerID  Level
1      Anita   NULL       1
2      Rahul   1          2
3      Sneha   1          2
4      Vikram  2          3
5      Pooja   2          3
6      Arun    4          4
```

- **Analytical applications:** CTEs are widely used for cohort and retention analysis (a cohort CTE joined to an activity CTE) and for year-over-year growth, where a monthly-sales CTE is followed by a CTE that uses LAG(total_sales, 12) to fetch last year's value.
- **Controlling recursion:** A recursive CTE can loop endlessly if the data contains a cycle. Limiting the depth with a condition such as WHERE Level < 10, or using the system recursion limit, prevents infinite loops and excessive processing.
- **Best practices:** Use descriptive names for CTEs and columns, split long calculations into separate CTEs, and avoid overuse because every extra step may affect performance. For logic reused often, consider indexed views or temporary tables.

Q21. Explain stored procedures and functions in PL/SQL. Write a procedure to raise the salary of an employee and return the new salary through an OUT parameter, and a function to calculate years of service from Employee(EmpID, Name, HireDate, Salary). Explain how program units are created, called, shared, debugged and tracked for dependencies.

Introduction: Stored procedures and functions are named PL/SQL blocks stored in the database. They move business logic from the application into the database, where it can be reused by many users and programs.

- **Named and anonymous units:** An anonymous block is unnamed and cannot be reused, whereas a named unit is stored and can be called by other programs and users. Server-side units are available from any connection but make the server do all the processing.
- **Creating a procedure:** The syntax is CREATE [OR REPLACE] PROCEDURE name (parameters) AS declarations BEGIN statements EXCEPTION handlers END. The CREATE PROCEDURE privilege is needed, and CREATE ANY PROCEDURE to create one in another schema.
- **Parameter modes:** IN passes a read-only value into the unit. OUT is a write-only value returned to the caller. IN OUT passes a value in and lets the unit change it and send it back. The default mode is IN.
- **Procedure example:** raise_salary receives the employee ID and the percentage as IN parameters and returns the updated salary through the OUT parameter p_new_sal using RETURNING ... INTO.

```
CREATE OR REPLACE PROCEDURE raise_salary(
        p_emp_id  IN  NUMBER,
        p_pct     IN  NUMBER,
        p_new_sal OUT NUMBER) AS
BEGIN
    UPDATE Employee
    SET Salary = Salary * (1 + p_pct / 100)
    WHERE EmpID = p_emp_id
    RETURNING Salary INTO p_new_sal;
    COMMIT;
END;
/
```

- **Calling a procedure:** In SQL*Plus a procedure is run with EXECUTE; from another unit it is called directly. Actual parameters are matched to formal ones by position. For employee 101 with salary 50000 and a 10 percent raise, the output is as shown.

```
DECLARE
    v_sal NUMBER;
BEGIN
    raise_salary(101, 10, v_sal);
    DBMS_OUTPUT.PUT_LINE('New salary: ' || v_sal);
END;
/
-- Output: New salary: 55000
```

- **Functions:** A function is created with CREATE OR REPLACE FUNCTION and must contain RETURN datatype in its header and a RETURN value statement in its body. It returns exactly one value and is called as part of an expression.

```
CREATE OR REPLACE FUNCTION service_years(p_hire_date IN DATE)
RETURN NUMBER IS
    v_years NUMBER;
BEGIN
    v_years := TRUNC(MONTHS_BETWEEN(SYSDATE, p_hire_date) / 12);
    RETURN v_years;
END;
/
SELECT Name, service_years(HireDate) AS Years FROM Employee;
-- An employee hired on 15-MAR-2019 shows 7 (as on Nov 2026)
```

- **Inline use and purity levels:** A function can be used inside SQL only if it has IN parameters, uses database datatypes and is stored in the database. It must also satisfy purity levels: WNDS (writes no database state), RNDS, WNPS and RNPS, with WNDS mandatory.
- **Sharing and privileges:** The owner gives access with GRANT EXECUTE ON unit_name TO username (or PUBLIC). Without it, users need the EXECUTE ANY PROCEDURE privilege. A function owned by another user is called as owner.function_name.

- **Debugging:** A compile error produces the message 'Procedure created with compilation errors'. The details are shown by the SHOW ERRORS command or by querying USER_ERRORS. DBMS_OUTPUT.PUT_LINE statements display variable values during execution.
- **Dependencies and recompilation:** A unit depends directly on objects it references and indirectly on objects those reference. USER_DEPENDENCIES, ALL_DEPENDENCIES and DBA_DEPENDENCIES list them, and DBMS_DDL.ALTER_COMPILE recompiles an invalidated unit.

Q22. A retail firm maintains Inventory(InvID, ItemName, QOH) and OrderLine(OrderID, InvID, OrderQty). Design database triggers to (i) reduce stock when an order line is inserted, (ii) adjust stock when the quantity is changed, (iii) restore stock when an order line is deleted, (iv) record every quantity change of more than 10 units in an audit table, and (v) block deletions on weekends. Explain the types of triggers, their firing order and how they are managed.

Introduction: A database trigger is a PL/SQL program unit that fires automatically on INSERT, UPDATE or DELETE events. It is used to keep related tables consistent, enforce business rules and audit changes.

- **Trigger versus procedure:** A trigger cannot accept input parameters and is never called explicitly. It executes only when its triggering event happens, which is called firing, and its execution is transparent to the user who ran the DML statement.
- **Privileges:** To create a trigger on a table you must own it, have the ALTER privilege on it or have ALTER ANY TABLE, and you also need the CREATE TRIGGER system privilege (CREATE ANY TRIGGER for another schema).
- **General syntax:** CREATE [OR REPLACE] TRIGGER name {BEFORE | AFTER | INSTEAD OF} {DELETE | INSERT | UPDATE [OF column]} ON table [FOR EACH ROW] [WHEN (condition)] followed by a PL/SQL block.
- **Types of triggers:** Triggers are classified by event (INSERT, UPDATE, DELETE), timing (BEFORE, AFTER, INSTEAD OF) and level (ROW or STATEMENT). Row-level triggers fire for every affected row, statement-level triggers once per statement, and INSTEAD OF triggers replace the action, mainly on views.
- **(i)-(iii) Stock maintenance:** One row-level trigger handles all three events. On INSERT the stock falls by the new quantity, on UPDATE the old quantity is returned and the new one deducted, and on DELETE the old quantity is restored. The WHERE clause is essential to change only the correct item.

```
CREATE OR REPLACE TRIGGER trg_orderline_stock
AFTER INSERT OR UPDATE OF OrderQty OR DELETE ON OrderLine
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE Inventory SET QOH = QOH - :NEW.OrderQty WHERE InvID = :NEW.InvID;
    ELSIF UPDATING THEN
        UPDATE Inventory SET QOH = QOH + :OLD.OrderQty - :NEW.OrderQty
        WHERE InvID = :NEW.InvID;
    ELSIF DELETING THEN
        UPDATE Inventory SET QOH = QOH + :OLD.OrderQty WHERE InvID = :OLD.InvID;
    END IF;
END;
/
-- QOH = 20: insert qty 3 -> 17; update qty to 5 -> 15; delete the line -> 20
```

- **:OLD and :NEW values:** In a row-level trigger :OLD holds the row before the change and :NEW the row after it. :OLD is empty for INSERT and :NEW is empty for DELETE. The predicates INSERTING, UPDATING and DELETING show which event fired the trigger.

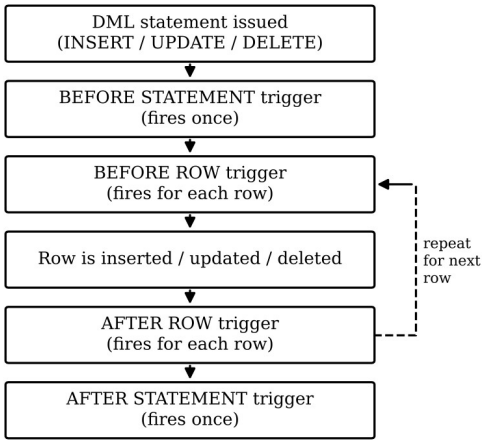
- **(iv) Audit with a WHEN clause:** The WHEN condition makes the trigger fire only for large changes, so small updates add no overhead. The trigger copies the old and new quantities with the date into an audit table.

```
CREATE OR REPLACE TRIGGER trg_orderline_audit
BEFORE UPDATE OF OrderQty ON OrderLine
FOR EACH ROW
WHEN (ABS(NEW.OrderQty - OLD.OrderQty) > 10)
BEGIN
    INSERT INTO OrderLine_Audit (OrderID, InvID, OldQty, NewQty, ChangedOn)
    VALUES (:OLD.OrderID, :OLD.InvID, :OLD.OrderQty, :NEW.OrderQty, SYSDATE);
END;
/
```

- **(v) Custom errors:** A statement-level BEFORE DELETE trigger checks the day and stops the operation with RAISE_APPLICATION_ERROR, which sets an error number between -20000 and -20999 and the message shown to the user. The delete is then rolled back.

```
CREATE OR REPLACE TRIGGER trg_orderline_del
BEFORE DELETE ON OrderLine
BEGIN
    IF TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN') THEN
        RAISE_APPLICATION_ERROR(-20001, 'Deletions are not allowed on weekends');
    END IF;
END;
/
```

- **Firing order and levels:** A statement-level trigger fires once per statement; a row-level trigger fires for each affected row. BEFORE row triggers can also set values, for example :NEW.UpperName := UPPER(:NEW.Name). The sequence is shown below.



- **Management and precautions:** Triggers are switched off or on with ALTER TRIGGER name DISABLE/ENABLE or ALTER TABLE name DISABLE ALL TRIGGERS, and removed with DROP TRIGGER name. USER_TRIGGERS lists them. Avoid COMMIT inside a trigger and keep the logic short.

Q23. A business wants to analyse its sales data stored in Sales(SaleID, CustomerID, Product, Category, SaleDate, Amount). Explain the concept and types of window functions and demonstrate, with queries, (i) the category total beside each sale, (ii) a running total, (iii) the top product in each category, and (iv) the month-over-month change in sales. Also explain how the gaps-and-islands problem is solved with window functions.

Introduction: A window function is applied to a set of rows defined by the OVER clause and returns a value for each row, so detail and analytical values appear together. Sample data used in this answer:

```
SaleID  CustomerID  Product  Category     SaleDate    Amount
1       C1          Laptop   Electronics  2026-01-10  60000
2       C2          Laptop   Electronics  2026-02-12  50000
3       C1          Phone    Electronics  2026-01-15  40000
4       C3          Phone    Electronics  2026-03-05  55000
5       C2          Shirt    Clothing     2026-01-20   4000
6       C4          Shirt    Clothing     2026-02-18   5000
7       C1          Jeans    Clothing     2026-03-09   9500
8       C3          Mixer    Appliances   2026-02-25  12000
9       C4          Fan      Appliances   2026-03-14   7000
10      C2          Fan      Appliances   2026-03-20   6000
```

- **Concept and OVER clause:** For every row, OVER creates a separate window that is independent of the windows of other rows. It may contain PARTITION BY, ORDER BY and a frame. Window functions let a developer move from iterative, row-by-row thinking to set-based thinking.
- **Types of window functions:** Aggregate functions (SUM, COUNT, MIN, MAX, AVG), ranking functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE), distribution or statistical functions (PERCENT_RANK, CUME_DIST, PERCENTILE_CONT, PERCENTILE_DISC) and offset functions (LAG, LEAD, FIRST_VALUE, LAST_VALUE).
- **(i) Partitioning:** PARTITION BY Category restricts each window to the rows of the same category, so SUM gives the category total on every row. The WHERE filter is applied first, so only the filtered rows form the windows.

```
SELECT SaleID, Category, Amount,
       SUM(Amount) OVER (PARTITION BY Category) AS CategoryTotal
FROM Sales
WHERE Category = 'Clothing';

SaleID  Category  Amount  CategoryTotal
5       Clothing  4000    18500
6       Clothing  5000    18500
7       Clothing  9500    18500
```

- **(ii) Ordering and framing:** Adding ORDER BY and the frame ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW makes the window grow from the first row to the current row, which produces a running total.

```
SELECT SaleDate, Amount,
       SUM(Amount) OVER (ORDER BY SaleDate, SaleID
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Sales
WHERE Category = 'Electronics';

SaleDate    Amount  RunningTotal
2026-01-10  60000   60000
2026-01-15  40000   100000
2026-02-12  50000   150000
2026-03-05  55000   205000
```

- **(iii) Ranking within groups:** RANK() with PARTITION BY Category orders the product totals inside each category. Because a window function cannot be filtered in WHERE, the ranking is placed in a derived table and the outer query keeps rank 1.

```
SELECT Category, Product, TotalSales, rnk
FROM (SELECT Category, Product, SUM(Amount) AS TotalSales,
             RANK() OVER (PARTITION BY Category
                          ORDER BY SUM(Amount) DESC) AS rnk
      FROM Sales
      GROUP BY Category, Product) AS ranked
WHERE rnk = 1;

Category     Product  TotalSales  rnk
Appliances   Fan      13000       1
Clothing     Jeans    9500        1
Electronics  Laptop   110000      1
```

- **(iv) Offset functions:** LAG returns a value from a previous row and LEAD from a following row. Applied to monthly totals, LAG gives the previous month so that growth or decline can be calculated without a self-join.

```
WITH MonthlySales AS (
    SELECT DATE_TRUNC('month', SaleDate) AS SaleMonth, SUM(Amount) AS Total
    FROM Sales GROUP BY DATE_TRUNC('month', SaleDate)
)
SELECT SaleMonth, Total,
       LAG(Total) OVER (ORDER BY SaleMonth) AS PrevTotal,
       Total - LAG(Total) OVER (ORDER BY SaleMonth) AS Change
FROM MonthlySales;

SaleMonth   Total   PrevTotal  Change
2026-01-01  104000  NULL       NULL
2026-02-01  67000   104000     -37000
2026-03-01  77500   67000      10500
```

- **Distribution functions:** PERCENT_RANK gives the relative rank of a row from 0 to 1, and CUME_DIST gives the fraction of rows with a value less than or equal to the current one. PERCENTILE_CONT interpolates a percentile, while PERCENTILE_DISC returns an existing value; they help to find medians and quartiles.
- **Position in query processing:** Window functions are evaluated after FROM, WHERE, GROUP BY and HAVING, so they work on the result of those phases. They are allowed only in the SELECT and ORDER BY clauses, which is why filtering on a window result needs a derived table or CTE.
- **Gaps and islands:** To find consecutive ranges, ROW_NUMBER() is subtracted from the value. Within an island both increase by 1, so the difference is constant and unique to that island. Grouping by the difference and taking MIN and MAX gives the start and end of each island.

```
-- T1(col1) holds 2, 3, 11, 12, 13, 27, 33, 34, 35, 42
SELECT MIN(col1) AS startrange, MAX(col1) AS endrange
FROM (SELECT col1, col1 - ROW_NUMBER() OVER (ORDER BY col1) AS grp
      FROM T1) AS d
GROUP BY grp
ORDER BY startrange;

startrange  endrange
2           3
11          13
27          27
33          35
42          42
```

- **Evolution and support:** Window functions were first standardised in an extension to SQL:1999 and enhanced up to SQL:2016. SQL Server added aggregates and ranking in 2005, frames, offset and statistical functions in 2012, and batch mode in 2016 and 2019. RANGE with INTERVAL, row-pattern recognition and the WINDOW clause are still missing.

Q24. A business wants to improve decisions using the data in Sales(SaleID, CustomerID, Product, Category, SaleDate, Amount). Develop an SQL-based analysis that retrieves relevant records, filters them using WHERE, groups them by Category, applies suitable aggregate functions and orders the summary. Also write queries for sales trend, customer segmentation, repeat customers and unusual sales days, and state how the information supports business analysis and how the insights should be communicated.

Introduction: Data analytics turns raw data into decisions through collection, processing, analysis, interpretation and communication; SQL supplies the analysis step through filtering, grouping, aggregation and window functions. Sample data:

```
SaleID  CustomerID  Product  Category     SaleDate    Amount
1       C1          Laptop   Electronics  2026-01-10  60000
2       C2          Laptop   Electronics  2026-02-12  50000
3       C1          Phone    Electronics  2026-01-15  40000
4       C3          Phone    Electronics  2026-03-05  55000
5       C2          Shirt    Clothing     2026-01-20   4000
6       C4          Shirt    Clothing     2026-02-18   5000
7       C1          Jeans    Clothing     2026-03-09   9500
8       C3          Mixer    Appliances   2026-02-25  12000
9       C4          Fan      Appliances   2026-03-14   7000
10      C2          Fan      Appliances   2026-03-20   6000
```

- **Analytics workflow:** Data collection leads to data processing, then analysis, interpretation and communication. Raw data has value only after it is interpreted in its business context, and SQL queries give precise results that can be explained to others.
- **Types of insights:** Descriptive insights show what happened, diagnostic insights explain why it happened, predictive insights estimate what may happen and prescriptive insights recommend what action to take. The queries below are mainly descriptive and diagnostic.
- **Category summary (WHERE, GROUP BY, aggregates, ORDER BY):** WHERE keeps the relevant records (sales above 5000), GROUP BY Category forms groups, COUNT, SUM and AVG summarise each group and ORDER BY sorts the summary so the strongest category appears first.

```
SELECT Category, COUNT(*) AS NoOfSales, SUM(Amount) AS TotalSales,
       ROUND(AVG(Amount), 2) AS AvgSale
FROM Sales
WHERE Amount > 5000
GROUP BY Category
ORDER BY TotalSales DESC;

Category     NoOfSales  TotalSales  AvgSale
Electronics  4          205000      51250.00
Appliances   3          25000       8333.33
Clothing     1          9500        9500.00
```

- **Trend analysis:** Grouping by month shows how sales move over time. Here January is 104000, February falls to 67000 and March recovers to 77500, which raises questions about seasonal effects or promotions in January.

```
SELECT DATE_TRUNC('month', SaleDate) AS SaleMonth, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY DATE_TRUNC('month', SaleDate)
ORDER BY SaleMonth;
-- Output: 2026-01-01 104000 | 2026-02-01 67000 | 2026-03-01 77500
```

- **Customer segmentation:** A CASE expression classifies customers by total spending so that marketing and resources can be targeted. Customers with high spending can receive loyalty offers while low spenders can be encouraged with discounts.

```
SELECT CustomerID, SUM(Amount) AS TotalSpent,
       CASE WHEN SUM(Amount) > 100000 THEN 'High spender'
            WHEN SUM(Amount) BETWEEN 50000 AND 100000 THEN 'Medium spender'
            ELSE 'Low spender' END AS SpendingCategory
FROM Sales
GROUP BY CustomerID;
-- Output: C1 109500 High | C2 60000 Medium | C3 67000 Medium | C4 12000 Low
```

- **Repeat purchases:** HAVING COUNT(SaleID) > 2 keeps only customers with repeated purchases, which indicates loyalty and helps to plan retention offers. Here customers C1 and C2 each made three purchases.

```
SELECT CustomerID, COUNT(SaleID) AS PurchaseCount
FROM Sales
GROUP BY CustomerID
HAVING COUNT(SaleID) > 2;
-- Output: C1 3 | C2 3
```

- **Anomaly detection:** The query compares each day's total with twice the average sale (2 x 24850 = 49700) and returns unusually high days. Their causes, such as promotions, seasonal demand or one-off bulk orders, are then investigated.

```
SELECT SaleDate, SUM(Amount) AS DailySales
FROM Sales
GROUP BY SaleDate
HAVING SUM(Amount) > (SELECT AVG(Amount) * 2 FROM Sales);
-- Output: 2026-01-10 60000 | 2026-02-12 50000 | 2026-03-05 55000
```

- **Pitfalls and missing data:** Analysts must avoid confirmation bias (using only data that supports a belief), overfitting (finding patterns that do not generalise) and treating correlation as causation. Missing values can be handled with COALESCE or by using only complete rows.
- **Communicating insights:** Bar charts suit comparisons, line graphs suit trends, scatter plots show relationships and heatmaps show intensity. Insights should be told as a story of problem, findings and recommendation, simplified for non-technical stakeholders.
- **Dashboards and performance:** Dashboards built in tools such as Tableau or Power BI give real-time KPI monitoring. For these queries to stay fast, indexes on SaleDate, CustomerID and Category are useful, and EXPLAIN should be used to check that index scans replace full table scans.